DSA0309 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 3 (CO3)

Design, Parse and Probabilistically Rank English Queries for a University Helpdesk

Programme: B.E. / B.Tech

Course Code: DSA0309

Course Name: Natural Language Processing

Nature: Individual

Duration: One Week

Total Marks: 50

Submission Date: 8-11-2026

Student Name: Totti Reddy Gayathri

Register Number: 192425239

Introduction

A university helpdesk receives messages about examinations, timetables, fees, hostels, library services and placements. Before the system can identify the meaning of a message, it must analyse its grammatical structure. This assignment develops a Context-Free Grammar (CFG), generates parse trees, checks agreement and verb subcategorization, and studies top-down, Earley and probabilistic parsing.

The assessment uses a controlled English fragment. The final design combines a feature-based grammar, Earley parsing and probabilistic ranking. The Day-4 challenge evaluates how the system can be extended to conversational and multi-clause messages.

TASK 1 – Design a Context-Free Grammar for English Syntax (5 Marks)

1.1 Formal definition

A Context-Free Grammar is represented as G = (N, Σ, R, S). N is the set of non-terminal symbols, Σ is the set of terminal symbols, R is the set of production rules and S is the start symbol. The start symbol is S.

CFG element	Definition in the helpdesk grammar
N – Non-terminals	S, DECL, IMP, YNQ, WHQ, NP, VP, PP, AP, CP, RelClause, Det, N, V, Pron, Aux, Adj, Adv, Prep, Conj, WhWord, Comp.

Σ – Terminals	The words listed in the lexicon in Section 1.3.
R – Rules	The production rules listed in Section 1.2.
S – Start symbol	S, representing a complete sentence or helpdesk query.

1.2 Chomsky-style phrase-structure rules

S → DECL | IMP | YNQ | WHQ

DECL → NP VP | NP Aux VP | DECL Conj DECL

IMP → VP

YNQ → Aux NP VP

WHQ → WhWord Aux NP VP | WhWord VP | WhWord NP Aux VP

NP → Pron | ProperN | Det N | Det Adj N | Det AP N

NP → NP PP | NP RelClause | NP Conj NP | Num N

VP → V | V NP | V NP NP | V PP | V NP PP | V CP

VP → Aux V | Aux VP | Adv VP | VP PP | VP Conj VP

PP → Prep NP | Prep Pron

AP → Adj | Adv Adj | Adj AP | Adj Conj Adj

CP → Comp DECL | Comp WHQ | Comp YNQ

RelClause → RelPron VP | RelPron Aux VP

DECL → Adv DECL

NP → Poss N | Det N Adv

VP → V NP CP | V Pron PP

The grammar contains more than 40 individual alternatives when the grouped alternatives are written as separate productions. The recursive rules NP → NP PP, NP → NP RelClause, VP → VP PP and DECL → DECL Conj DECL allow phrases and clauses to grow to different lengths. This gives the grammar productive coverage without writing a separate rule for every possible sentence length.

1.3 Lexicon: 60+ helpdesk words

Part of speech	Words
Determiners	the, a, an, this, that, these, those, my, your
Nouns	student, students, staff, teacher, teachers, exam, exams, fee, fees, hostel, room, library, book, books, timetable, class, form, notice, receipt,

	portal, placement, result, results, scholarship, course, department, campus, application, hall, ticket, payment, date, query, service, office, document, assignment, subject, laboratory, seminar, schedule, account
Pronouns	I, we, you, he, she, they, it, me, us, him, her, them
Proper nouns	Monday, English, Python, India
Verbs	show, submit, send, tell, book, check, apply, need, want, open, close, attend, pay, print, download, upload, visit, issue, display, confirm, explain, provide, help, arrive, crash, begin, postpone, hold, be, is, are, was, were, has, have, do, does, can, will, should
Adjectives	late, available, revised, pending, important, new, old, final, online, academic, postponed
Adverbs	today, tomorrow, quickly, recently, soon
Prepositions	in, on, at, near, for, from, to, with, about, after, before, during, by
Conjunctions	and, or, but
Wh-words / complementizers	what, when, where, who, which, how, whether, that, if

The lexicon is separate from the grammar rules. The categories are kept distinct because each part of speech has a different syntactic role. Verbs are further divided by complement pattern in Task 5.

1.4 Constituency, head and phrasal category

A constituent is a group of words that behaves as a single unit. A phrase is named after its head. The head of a noun phrase is a noun, while the head of a verb phrase is a verb. For example, students is the head of the NP the students, and submit is the head of the VP submit the form.

Constituency test	Helpdesk example	Interpretation
Substitution	The students checked the timetable → They checked the timetable.	The students can be replaced by they; it forms an NP.

Movement	The notice near the library was displayed → Near the library, the notice was displayed.	Near the library moves as one PP constituent.
Coordination	The student checked the fee and the timetable.	The two NPs can be coordinated with and.

The grammar is context-free by construction because every production replaces one non-terminal with a fixed sequence of terminals and/or non-terminals, independent of surrounding symbols. English agreement across long distances and cross-serial dependencies are examples of phenomena that are difficult to represent compactly with a plain CFG. Feature constraints are used to improve agreement handling.

TASK 2 – Context-Free Rules, Derivations and Parse Trees (3 Marks)

2.1 Five sentences from the query set

1. The student checks the timetable.
2. Students submit the forms.
3. Show the exam result.
4. Can the teacher confirm the date?
5. Where is the library?

2.2 Leftmost and rightmost derivations

Worked sentence: The student checks the timetable.

Leftmost derivation:

S ⇒ DECL ⇒ NP VP ⇒ Det N VP ⇒ The student VP ⇒ The student V NP ⇒ The student checks Det N ⇒ The student checks the timetable

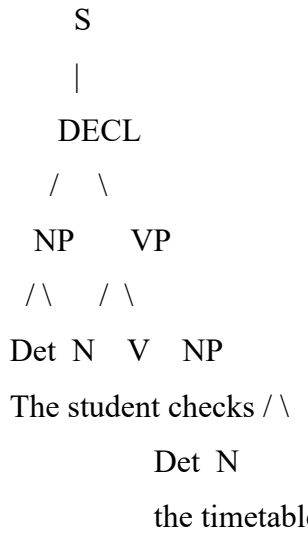
Rightmost derivation:

S ⇒ DECL ⇒ NP VP ⇒ NP V NP ⇒ NP V Det N ⇒ NP V the timetable ⇒ Det N checks the timetable ⇒ The student checks the timetable

Both derivations produce the same tree. The leftmost derivation always expands the leftmost non-terminal, while the rightmost derivation expands the rightmost non-terminal.

2.3 Bracketed notation and tree diagram

[S [DECL [NP [Det The] [N student]] [VP [V checks] [NP [Det the] [N timetable]]]]]



For the displayed tree, height is 4 edges from S to a terminal. The node count is 11 when the root, phrase nodes, part-of-speech nodes and word nodes are counted. The derivation shown uses 7 production applications. These measures depend on the exact tree convention used.

2.4 Structural ambiguity: two sentences

Sentence A: Show the students with the late fee notice. In the first analysis, the PP with the late fee notice attaches to the NP students and identifies which students are meant. In the second analysis, it attaches to the VP show and describes the showing event.

[S [VP [V Show] [NP [NP the students] [PP with [NP the late fee notice]]]]]

[S [VP [VP [V Show] [NP the students]] [PP with [NP the late fee notice]]]]

Sentence B: Book the room near the library for the seminar. The PP near the library may modify the noun room or the booking VP. The phrase for the seminar may also attach to the NP or VP, depending on the intended interpretation.

[S [VP Book [NP [NP the room] [PP near the library]] [PP for the seminar]]]

[S [VP [VP Book [NP the room]] [PP near the library] [PP for the seminar]]]

A derivation is the sequence of rule applications used to generate a sentence. A parse tree is the final hierarchical representation. Different derivation orders can produce the same tree. A string belongs to L(G) if at least one complete derivation from S generates it. Membership testing tells whether a parse exists, but it does not necessarily identify the intended meaning.

TASK 3 – Sentence-Level Constructions (4 Marks)

3.1 Rules for four main constructions

S → DECL | IMP | YNQ | WHQ

DECL → NP VP

IMP → VP

YNQ → Aux NP VP

WHQ → WhWord Aux NP VP

WHQ → WhWord VP

WHQ → WhWord NP Aux VP

DECL → DECL Conj DECL

NP → NP RelClause

RelClause → RelPron Aux VP

CP → Comp DECL | Comp WHQ | Comp YNQ

Construction	Example	Basic structure
Declarative	The student checks the timetable.	S → NP VP
Imperative	Submit the form.	S → VP
Yes/no question	Can the teacher confirm the date?	S → Aux NP VP
Non-subject wh-question	When will the exams begin?	S → WhWord Aux NP VP
Subject wh-question	Who submitted the form?	S → WhWord VP

3.2 Auxiliary inversion, wh-movement and clauses

In a yes/no question, the auxiliary moves before the subject. The declarative order is The teacher can confirm the date, while the question order is Can the teacher confirm the date? A wh-question places a wh-word at the beginning. In What did the student submit __?, the blank represents the object gap associated with what.

A relative clause modifies a noun, as in the exams that were postponed. A subordinate clause appears inside another clause, as in The student asked whether the fee was paid. Coordination joins phrases or clauses, as in fees and hostel charges. Each construction adds rules and possible attachment positions. Long-distance dependencies are difficult to represent with plain CFG rules because the grammar must connect a moved phrase with its gap.

TASK 4 – Agreement (3 Marks)

4.1 Agreement phenomena

Subject–verb agreement requires the verb to match its subject in person and number. Determiner–noun agreement requires compatible number, as in this form and these forms. Pronoun case requires subject pronouns such as she and they in subject position and object pronouns such as her and them in object position.

4.2 Agreement by multiplying categories

S → NP-SG VP-SG | NP-PL VP-PL

NP-SG → Det-SG N-SG | Pron-SG

NP-PL → Det-PL N-PL | Pron-PL

VP-SG → V-SG | V-BASE NP

VP-PL → V-BASE | V-BASE NP

For this simplified design, separate singular and plural categories approximately double the relevant NP, VP and sentence rule families. If person has 3 values, number 2, case 2 and gender 3, the complete feature combination count is $3 \times 2 \times 2 \times 3 = 36$. A rule family that explicitly enumerates every combination could therefore require up to 36 variants. This is rule explosion. The exact number of grammar productions depends on how categories are grouped.

4.3 Agreement test results

Sentence	Baseline CFG	Agreement-aware CFG
The student submits the form.	Accept	Accept
The students submit the forms.	Accept	Accept
She checks the timetable.	Accept	Accept
They check the timetable.	Accept	Accept
This form is available.	Accept	Accept
These forms are available.	Accept	Accept
The students submits the form.	May accept	Reject
These form is available.	May accept	Reject
This forms are available.	May accept	Reject
Her checks the timetable.	May accept	Reject in subject position
They checks the timetable.	May accept	Reject
The student submit the form.	May accept	Reject

The baseline CFG may accept incorrect combinations if it does not encode number and case. The agreement-aware grammar rejects these examples. Feature structures in Task 8 provide a more compact method.

TASK 5 – Subcategorization (4 Marks)

5.1 Definition and five verb frames

Subcategorization, also called valence, describes the complements a verb requires or permits. Different verbs select different complement patterns.

Frame	Pattern	Example
Intransitive	V	The portal crashed.
Transitive	V + NP	Submit the form.
Ditransitive	V + NP + NP	Send me the receipt.
Sentential complement	V + CP	Tell me that the fee is paid.
PP complement	V + PP	Apply for a hostel room.
Object + PP	V + NP + PP	Provide the student with a receipt.

5.2 Encoding with verb categories

VP → V-INTR

VP → V-TRANS NP

VP → V-DITRANS NP NP

VP → V-COMP CP

VP → V-PP PP

VP → V-OBJ-PP NP PP

The lexicon places each verb in a category that matches its complement frame. This prevents a verb from taking an incorrect number or type of complements. The disadvantage is that a verb with more than one frame must appear in multiple categories, making the lexicon larger.

5.3 Complements and adjuncts

A complement is selected by the verb and helps complete its meaning. In Apply for a hostel room, the PP for a hostel room is selected by apply. An adjunct adds optional information. In The student submitted the

form yesterday, yesterday is an optional time adjunct. Removing an adjunct leaves a complete sentence, while removing a required complement may make the sentence incomplete.

Sentence	Expected result	Reason
The portal crashed.	Accept	Crash is intransitive.
Submit the form.	Accept	Submit takes an object NP.
Send me the receipt.	Accept	Send takes two NP complements.
Tell me that the fee is paid.	Accept	Tell can take a clause complement.
Apply for a hostel room.	Accept	Apply takes a selected PP.
Submit.	Reject in the transitive use	Required object is missing.
The portal crashed the form.	Reject	Crash does not take this direct object frame.

A verb that permits multiple frames can create multiple analyses. Lexical subcategorization filters out invalid structures, but additional frames may increase ambiguity and parsing cost.

TASK 6 – Top-Down Parsing (4 Marks)

6.1 Recursive-descent algorithm

A top-down parser begins with S and expands non-terminals until its predicted terminals match the input. It is goal-driven and uses backtracking when an alternative fails.

```
TOP-DOWN-PARSE(input):
  agenda ← [S]
  choose the leftmost non-terminal A
  for each production A → β:
     replace A with β
     match terminal symbols with the next input words
     if a mismatch occurs, restore the previous agenda and backtrack
  accept if all input words are matched and no non-terminals remain
  otherwise reject
```

6.2 Trace on an unambiguous sentence

Input: The student checks the timetable.

Step	Agenda / goal	Operation
1	S	Expand S → DECL
2	DECL	Expand DECL → NP VP
3	NP VP	Expand NP → Det N
4	Det N VP	Match The, then student
5	VP	Expand VP → V NP
6	V NP	Match checks
7	NP	Expand NP → Det N
8	Det N	Match the, then timetable; accept

6.3 Trace on an ambiguous sentence

Input: Show the students with the notice. The parser first chooses NP → NP PP, attaching with the notice to students. This gives the meaning that the students have the notice. If the parser instead needs the PP to modify show, it backtracks and uses VP → VP PP. The two successful paths produce different trees.

6.4 Left recursion, repeated work and complexity

The rule NP → NP PP is left-recursive. A simple recursive-descent parser can call NP again before consuming an input word and may loop indefinitely. One solution is to rewrite the rule as NP → BaseNP PP* or to use a left-corner parser. This changes the internal search strategy but the final tree can still show repeated PP attachments.

A naive top-down parser can parse the same substring repeatedly. With k PP phrases that each have two possible attachment sites, a rough number of attachment combinations is 2^k. Therefore, the worst-case search can be exponential for an ambiguous grammar. Memoization and chart parsing store repeated subproblems and reduce this waste.

TASK 7 – Earley Parsing (5 Marks)

7.1 Earley state and three operations

An Earley state is written [A → α • β, i]. A is the left-hand side, α is the part already recognised, β is the part still expected and i is the input position where the rule began.

PREDICTOR adds possible productions when the symbol after the dot is a non-terminal. SCANNER matches the next input word when the symbol after the dot is a terminal. COMPLETER advances a waiting state when a predicted constituent has been completed.

7.2 Hand simulation and chart

Input: The student checks the timetable. Token positions are 0 The, 1 student, 2 checks, 3 the, 4 timetable and 5 end. The chart below shows the main states for the small grammar S → DECL; DECL → NP VP; NP → Det N; VP → V NP; Det → The | the; N → student | timetable; V → checks.

Chart column	Main states	Operation that adds the states
S[0]	S → • DECL,0; DECL → • NP VP,0; NP → • Det N,0; Det → • The,0	Predictor
S[1]	Det → The •,0; NP → Det • N,0; N → • student,1	Scanner, then Predictor
S[2]	N → student •,1; NP → Det N •,0; DECL → NP • VP,0; VP → • V NP,2	Completer, then Predictor
S[3]	V → checks •,2; VP → V • NP,2; NP → • Det N,3	Scanner, then Predictor
S[4]	Det → the •,3; NP → Det • N,3; N → • timetable,4	Scanner, then Predictor
S[5]	N → timetable •,4; NP → Det N •,3; VP → V NP •,2; DECL → NP VP •,0; S → DECL •,0	Completer

The completed state S → DECL •,0 in S[5] confirms that the complete input has been recognised. A full implementation stores back-pointers showing which earlier states and words created each state. These pointers recover the parse tree. For ambiguous PP attachment, the chart stores both analyses while sharing common partial results. The chart above is a compact hand trace of the principal states; an implementation may include additional predictor states for every lexical alternative.

7.3 Left recursion and complexity

Earley parsing handles left recursion because it stores states in chart columns and avoids processing an identical state repeatedly. An ambiguous attachment creates alternative states and back-pointers rather than requiring a separate full parse from the beginning.

For a general CFG, Earley parsing has $O(n^3)$ worst-case time and $O(n^2)$ chart space for an input of n words. In practical cases it may be faster. A top-down parser may revisit the same subproblem, whereas Earley stores the state in the chart. The number of states depends on the grammar and input, so measured counts should be reported from the implementation.

TASK 8 – Feature Structures and Unification (4 Marks)

8.1 Attribute–value structures

A feature structure represents grammatical information as attribute–value pairs. NUM represents number, PER represents person, CASE represents grammatical case and SUBCAT represents a verb's complement requirements. A feature path is a route to a value, such as NP.NUM. Reentrancy means that two paths share one value or structure.

Lexical entry	Feature structure
student	[CAT:N, NUM:SG, PER:3]
students	[CAT:N, NUM:PL, PER:3]
she	[CAT:PRON, NUM:SG, PER:3, CASE:NOM]
her	[CAT:PRON, NUM:SG, PER:3, CASE:ACC]
submit	[CAT:V, SUBCAT:⟨ NP⟩ , AGR:3SG/BASE]
crash	[CAT:V, SUBCAT:⟨ ⟩ , AGR:3SG/BASE]
send	[CAT:V, SUBCAT:⟨ NP,NP⟩]

8.2 Unification and subsumption

Unification combines compatible feature structures. If the same feature has conflicting values, unification fails. Subsumption means that a more general feature structure is compatible with a more specific one.

Unification	Result
[NUM:SG] with [PER:3]	Success: [NUM:SG, PER:3]
[CASE:NOM] with [CASE:NOM, PER:3]	Success: [CASE:NOM, PER:3]
[NUM:SG] with [NUM:PL]	Failure because the number values conflict.

8.3 Feature-based grammar rules

S → NP VP, with NP.NUM = VP.SUBJECT-NUM

NP → Det N, with Det.NUM = N.NUM

NP → Pron, with Pron.CASE = NOM in subject position

VP → V NP, with V.SUBCAT = ⟨ NP⟩

VP → V NP NP, with V.SUBCAT = ⟨ NP,NP⟩

VP → V CP, with V.SUBCAT = ⟨ CP⟩

PP → Prep NP, with the preposition selecting an NP

In The student submits the form, the subject NP has NUM:SG and the verb submits has third-person singular agreement. Unification succeeds. In The students submits the form, the subject has NUM:PL but the verb requires 3SG, so unification fails. The same method checks determiner–noun number and whether a verb receives the required complement.

Feature structures replace many duplicated category rules with shared attributes. During Earley COMPLETER, the completed constituent's features can be unified with the waiting state. This introduces extra unification work, and complex feature structures can increase parsing cost.

TASK 9 – Probabilistic Context-Free Grammars (5 Marks)

9.1 PCFG definition and normalisation

A PCFG assigns a probability P(A → β | A) to every production. The probabilities of all productions with the same left-hand side must sum to 1. The probability of a parse tree is the product of the probabilities of all productions used in that tree.

9.2 Mini-treebank counts and maximum-likelihood estimates

The following is an illustrative 20-sentence hand-annotated mini-treebank summary. For each left-hand side, the probability is Count(rule) divided by the total count of rules with that left-hand side. The counts are grouped by left-hand side, and each group is normalised separately.

Rule	Count	LHS total	MLE probability
S → DECL	12	20	0.60
S → IMP	3	20	0.15
S → YNQ	3	20	0.15
S → WHQ	2	20	0.10

DECL → NP VP	18	20	0.90
DECL → NP Aux VP	2	20	0.10
NP → Det N	12	20	0.60
NP → Pron	5	20	0.25
NP → NP PP	3	20	0.15
VP → V NP	10	20	0.50
VP → V	4	20	0.20
VP → V PP	3	20	0.15
VP → VP PP	3	20	0.15

Example: P(S → DECL) = 12/20 = 0.60. The probabilities for each listed left-hand side sum to 1. An unseen rule receives zero under unsmoothed MLE. Add-one smoothing or a small reserved probability mass can assign non-zero probability to unseen rules; the probabilities must then be renormalised. The table is an illustrative estimate and should be replaced by counts from the student's own annotated sentences if an actual treebank is prepared.

9.3 Probability of ambiguous parse trees

For the PP attachment example, assume the relevant probabilities are P(S → IMP)=0.15, P(IMP → VP)=1.0, P(VP → V NP)=0.50, P(VP → VP PP)=0.15, P(NP → Det N)=0.60 and P(NP → NP PP)=0.15. For a short demonstration, lexical production probabilities are set to 1.

Tree A attaches the PP inside the NP. Its simplified probability is 0.15 × 1.0 × 0.50 × 0.15 × 0.60 = 0.00675. Tree B attaches the PP to the VP. Its simplified probability is 0.15 × 1.0 × 0.15 × 0.50 × 0.60 = 0.00675. These illustrative values give a tie. A real treebank may assign different probabilities.

The sentence probability is the sum of the probabilities of all its parse trees. If these are the only two trees, P(sentence) = 0.00675 + 0.00675 = 0.0135. The Viterbi parse is the individual tree with the highest probability. These are partial illustrative products; a complete PCFG calculation must include every rule in each complete tree.

9.4 CNF conversion and probabilistic CKY

Chomsky Normal Form uses binary productions A → B C and lexical productions A → word, apart from an optional start-symbol exception. A longer rule such as VP → V NP PP can be binarised by introducing X: VP → V X and X → NP PP. Terminals in longer productions are replaced with pre-terminal symbols.

for span length = 1 to n:

```
  for each start position i:
    for each split point k:
      for each binary rule A → B C:
        score = P(A → B C) × chart[i,k,B] × chart[k,j,C]
        keep the maximum score and its back-pointer
  return the highest-scoring S covering the full sentence
```

The CKY table for a particular sentence depends on the complete CNF grammar and lexical probabilities. The above pseudocode shows the Viterbi calculation; actual numeric cells must be filled from the chosen sentence and complete rule table.

9.5 PCFG assumptions and limitations

A PCFG assumes that a rule probability depends on its left-hand-side category and not on the full surrounding context. The same rule receives the same probability in different sentence positions. This position-invariance can be inaccurate. A PCFG may also prefer a frequent or shorter analysis that does not match the intended meaning. It has limited lexical sensitivity because it may not know that a particular verb strongly prefers a particular preposition. Lexicalised PCFGs or neural reranking can reduce these limitations.

TASK 10 – Implement and Test the Parsers (3 Marks)

10.1 Python implementation

Install NLTK by running pip install nltk. The following starter program builds a small CFG and runs a recursive-descent top-down parser and an Earley chart parser.

```
from nltk import CFG
from nltk.parse import RecursiveDescentParser, EarleyChartParser

grammar = CFG.fromstring('''
S -> DECL | IMP
DECL -> NP VP
IMP -> VP
NP -> DET N | NP PP
VP -> V NP | V | VP PP
PP -> P NP
DET -> 'the' | 'a'
N -> 'student' | 'form' | 'timetable' | 'library'
```

```
V -> 'checks' | 'submit' | 'show'
P -> 'near' | 'with'
''')

tokens = 'the student checks the timetable'.split()
parsers = [RecursiveDescentParser(grammar), EarleyChartParser(grammar)]
for parser in parsers:
    trees = list(parser.parse(tokens))
    print(type(parser).__name__, 'number of parses:', len(trees))
    for tree in trees:
        print(tree)
```

This is a working starter for a small fragment, not the complete 40-plus-rule grammar. The full implementation must add the remaining rules, feature checks and PCFG/Viterbi ranking. Run all 30 test sentences and record the actual results on the computer used. Do not invent runtime or state counts.

10.2 Thirty-sentence test set

No.	Test sentence	Construction
1	The student checks the timetable.	Declarative
2	The students submit the forms.	Declarative
3	The teacher confirms the exam date.	Declarative
4	The portal crashed.	Declarative
5	The library is open.	Declarative
6	The hostel fee is pending.	Declarative
7	Submit the application.	Imperative
8	Show the exam result.	Imperative
9	Check the timetable.	Imperative
10	Book the room.	Imperative
11	Print the hall ticket.	Imperative
12	Download the receipt.	Imperative
13	Can the teacher confirm the date?	Yes/no question

14	Does the student need a form?	Yes/no question
15	Are the library books available?	Yes/no question
16	Will the exam begin tomorrow?	Yes/no question
17	Where is the library?	Wh-question
18	When will the exams begin?	Wh-question
19	Which form should I submit?	Wh-question
20	Who checked the result?	Subject wh-question
21	Show the students with the notice.	Ambiguous PP attachment
22	Book the room near the library for the seminar.	Ambiguous PP attachment
23	The student with the receipt paid the fee.	PP attachment
24	The teacher saw the student with the form.	PP attachment
25	Fees and hostel charges are pending.	Coordination
26	The exams that were postponed will begin soon.	Relative clause
27	Tell me whether the fee is paid.	Embedded clause
28	The students submits the form.	Agreement error
29	These form is available.	Agreement error
30	The portal crashed the form.	Subcategorization error

10.3 Results recording format

Sentence no.	Parser	Parse found?	No. of parses	States / steps	Runtime (ms)
1	Top-down	To be measured	To be measured	To be measured	To be measured
1	Earley	To be measured	To be measured	To be measured	To be measured
1	PCFG/Viterbi	To be	Best parse	To be	To be

		measured	score	measured	measured
2	Top-down / Earley / PCFG	Fill after execution	Fill after execution	Fill after execution	Fill after execution

Test at least one left-recursive structure, ambiguous PP attachment, ungrammatical agreement examples and a long sentence up to 20 words. Compare the program's output with the hand calculations in Tasks 7 and 9. A plot should show sentence length on the x-axis and the number of chart states on the y-axis. Use measured values from the program.

TASK 11 – Compare, Recommend and Defend the Parsing Design (2 Marks)

Factor	Top-down parser	Earley parser	PCFG / Viterbi
Coverage	Depends on the grammar and rule order.	Handles general CFGs, including left recursion.	Ranks the parses generated by the grammar.
Ambiguity	May backtrack through alternatives.	Stores alternatives in the chart.	Chooses the highest-probability parse.
Cost	Can repeat work; exponential worst case for ambiguous grammars.	$O(n^3)$ worst-case time and $O(n^2)$ chart space.	Uses chart parsing plus probability calculations.
Maintenance	Simple for a small controlled grammar.	Rules can be extended while retaining chart parsing.	Requires annotated data and probability updates.

Recommended design: a feature-based CFG with an Earley parser and a PCFG ranking layer. Feature unification checks subject–verb agreement, determiner–noun agreement, pronoun case and verb subcategorization. Earley parsing finds valid structures while avoiding repeated processing of identical states. PCFG ranking selects the most probable tree when multiple parses are available.

Manual effort includes grammar design, lexicon preparation, sentence annotation, parser implementation and testing. A hand-written grammar is suitable for a limited domain where rules must be explainable. As the number and variety of messages grow, collecting annotated examples and using statistical ranking becomes more economical than continuously adding special-case rules.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The new helpdesk must accept conversational, multi-clause and partly ungrammatical messages. These may include greetings, embedded questions, relative clauses, coordinated clauses, fragments, ellipsis and missing punctuation. The original assumptions of controlled vocabulary, fully grammatical input, low ambiguity and manually written coverage are no longer reliable.

Longer messages create more possible phrase attachments and clause combinations. If several PPs can attach to different phrases, the number of candidate analyses can grow rapidly. Earley parsing reduces repeated work by sharing chart states, but the chart still becomes larger as the sentence and grammar grow.

B. Evaluation of the Original Design (2 Marks)

New message	**Likely result using original grammar**	**First unsupported feature**
Hi, can you tell me when the exams that were postponed will be held?	May fail or return only a partial parse.	Greeting, embedded wh-question and relative clause.
Can you tell me whether the hostel fee is due?	May fail if embedded question rules are absent.	Embedded whether-clause.
The exam is postponed and I need to know the new date.	May fail or parse only one clause.	Coordination of clauses and conversational wording.

The top-down parser may try many alternatives before failing at an unsupported construction. Earley parsing avoids duplicate states but cannot analyse a rule that is not present. The PCFG cannot rank a complete parse if the grammar produces none. This demonstrates that grammar coverage must be improved before probabilistic ranking can help.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original design. The original CFG is useful for the university domain, so it should be extended rather than discarded. Add rules for greetings, embedded clauses, relative clauses, coordinated clauses and fragments. Use feature structures to pass agreement and gap information across clause boundaries.

S → Greeting S | DECL | IMP | YNQ | WHQ | S Conj S

CP → Comp DECL | Comp YNQ | Comp WHQ

DECL → NP VP | DECL Conj DECL

NP → NP RelClause

RelClause → RelPron VP | RelPron Aux VP

WHQ → WhWord Aux NP VP | WhWord VP

VP → V NP CP | Aux VP

FRAG → NP | PP | WhWord

The revised design keeps Earley parsing as the main parser, adds feature unification for agreement and subcategorization, and estimates a larger PCFG from real helpdesk messages. A robust partial-parsing fallback identifies phrases even when a complete sentence cannot be parsed. The system can ask a clarification question when a required phrase is missing.

Coverage is expected to improve because more patterns are supported. Ambiguity may also increase because the grammar accepts more structures. Earley parsing controls repeated computation, and PCFG ranking helps choose a likely analysis. Annotation and maintenance effort increase because more real messages must be collected and labelled.

D. Evidence-Based Defence (2 Marks)

The original CFG handles basic declaratives, imperatives and questions. Derivations and parse trees show how phrase structures are generated. Agreement tests show why number and case information are needed. Subcategorization tests show that verbs select different complement patterns. The top-down trace demonstrates backtracking, while the Earley chart demonstrates shared states. The PCFG calculation shows how probabilities can rank ambiguous trees.

The changed requirement introduces constructions that the original controlled grammar does not fully cover. Retaining the original design without changes would cause parsing failures. Replacing everything would discard useful grammar rules and require more effort. Modifying the CFG and adding a partial-parsing fallback is therefore the most balanced decision. The feature-based CFG, Earley parser and PCFG ranking layer improve coverage and interpretation while preserving explainability.

Remaining limitations include unseen words, slang, spelling mistakes, code-mixing, long-distance dependencies and ambiguity that cannot be resolved by syntax alone. The system should ask clarification questions and continue collecting annotated examples.

Conclusion

This assessment designed a CFG for university helpdesk queries and explained derivations, parse trees, sentence constructions, agreement and verb subcategorization. It examined top-down parsing, Earley parsing, feature structures and PCFGs. The recommended design is a feature-based CFG with Earley

parsing and probabilistic ranking. Under the Day-4 requirement, the grammar should be modified and supported by partial parsing and additional annotated data.

References

Jurafsky, D. and Martin, J. H., Speech and Language Processing, sections on syntax, parsing and probabilistic grammars.

Allen, J., Natural Language Understanding, sections on grammar and parsing.

NLTK Documentation, Parsing and Probabilistic Parsing, https://www.nltk.org/.

DSA03 Natural Language Processing, CO3 Assignment 3 question and rubric.